POSTGRESQL 16 · NODE.JS

Cómo aplico la lógica de datos en PostgreSQL

Sustentación de base de datos de **Memoria Ninja**: cada concepto explicado con el objeto real del proyecto (código + lo que digo).

- 1. Cómo se aplica el **ACID**
- 2. Cómo aplica **CAP**
- 3. Las **4 vistas**
- 4. Los **2 procedimientos almacenados**
- 5. El **trigger**

Todo sale de sql/esquema.sql

01 · CÓMO SE APLICA

ACID en mi base de datos

Las cuatro garantías las sostiene el motor, no el código de la aplicación.

✅ **Sí se aplica, y correctamente.** Cada letra está cubierta por un objeto concreto del proyecto y comprobada por pruebas.

A · ATOMICIDAD

CÓDIGO

```
-- database.transaccion(): una conexión dedicada
BEGIN  ->  ...sentencias...  ->  COMMIT
                     (si algo falla ->  ROLLBACK)
```

LO QUE DIGO

Un procedimiento como **sp_crear_partida_completa** hace dos inserciones y, si una falla, PostgreSQL revierte las dos: o todo, o nada. Eso lo garantiza la transacción, que se ejecuta sobre una conexión dedicada (`BEGIN → COMMIT / ROLLBACK`).

C · CONSISTENCIA

CÓDIGO

```
CONSTRAINT fk_historial_usuario
  FOREIGN KEY (id_usuario) REFERENCES usuarios (id);
CHECK (correo ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$')
```

LO QUE DIGO

La base impone las reglas: clave foránea del historial hacia usuarios, UNIQUE de usuario/correo, y un CHECK con forma de correo. Por eso un resultado para un usuario inexistente se rechaza (error de clave foránea).

I · AISLAMIENTO

CÓDIGO

```
... FOR UPDATE;   -- bloquea la fila mientras dura la transacción
```

LO QUE DIGO

> Con **SELECT … FOR UPDATE** dos operaciones concurrentes sobre la misma partida no se pisan, y un código de verificación no se puede usar dos veces. Se usa al leer-modificar-guardar la partida y en `fn_confirmar_codigo`.

D · DURABILIDAD

CÓDIGO

```
SELECT estado_json FROM partidas WHERE id = ?;
```

LO QUE DIGO

> La escritura se confirma en disco (WAL) y la lectura vuelve a la tabla, no a un caché en memoria. Por eso una partida sigue existiendo después de reiniciar el servidor.

02 · CÓMO APLICA

CAP en mi proyecto

El teorema describe sistemas distribuidos con réplicas.

✅ **CAP no se "aplica" porque no corresponde:** el proyecto es un sistema **CA**, y eso no es un defecto.

EL TIPO DE SISTEMA

LO QUE DIGO

> Memoria Ninja es un **monolito**: un servidor Node.js y **una sola instancia** de PostgreSQL, sin réplicas. La **P** de CAP (tolerancia a particiones) describe cortes de red *entre nodos que replican datos*; al haber un solo nodo, esa partición no existe.

POR QUÉ NO HAY DISYUNTIVA

LO QUE DIGO

> Como no hay réplicas, no hay que elegir entre consistencia y disponibilidad: el sistema es **consistente** (ACID del punto 1) y **disponible** mientras el nodo esté en pie. Recién si algún día agrego réplicas distribuidas, el teorema empezaría a aplicar.

03 · ABSTRACCIÓN DE LECTURA

Las 4 vistas

JOIN, GROUP BY y los cálculos viven en PostgreSQL, no en el código.

El backend solo hace `SELECT * FROM <vista>`. La lógica relacional queda definida una sola vez en la base.

VISTA 1 · VW_PARTIDAS_RECIENTES

CÓDIGO

```
CREATE OR REPLACE VIEW vw_partidas_recientes AS
SELECT id, creada_en, actualizada_en
FROM partidas
ORDER BY actualizada_en DESC, creada_en DESC;
```

LO QUE DIGO

> Muestra las partidas ordenadas por la más reciente. Es la vista que aprovecha el trigger del punto 5: `actualizada_en` la llena el trigger, y acá se usa para ordenar.

VISTA 2 · VW_HISTORIAL_DETALLADO

CÓDIGO

```
CREATE OR REPLACE VIEW vw_historial_detallado AS
SELECT h.id, h.id_usuario, u.usuario, h.id_partida,
       h.zona, h.puntaje, h.modo, h.ganador, h.jugada_en
FROM historial_partidas h
LEFT JOIN usuarios u ON u.id = h.id_usuario
ORDER BY h.jugada_en DESC;
```

LO QUE DIGO

> El historial guarda el `id_usuario`, pero para mostrar el **nombre** del jugador hay que cruzar con la tabla `usuarios`. Ese JOIN lo resuelve la vista; el backend ya recibe el nombre listo.

VISTA 3 · VW_RANKING_MEJOR_PUNTAJE

CÓDIGO

```sql
CREATE OR REPLACE VIEW vw_ranking_mejor_puntaje AS
SELECT u.id AS id_usuario, u.usuario,
       MAX(h.puntaje)  AS mejor_puntaje,
       COUNT(h.id)     AS total_partidas,
       MAX(h.jugada_en)AS ultima_jugada
FROM historial_partidas h
INNER JOIN usuarios u ON u.id = h.id_usuario
GROUP BY u.id, u.usuario
ORDER BY mejor_puntaje DESC;
```

LO QUE DIGO

Es el ranking. Con **MAX** saco el mejor puntaje y con **COUNT** cuántas partidas jugó; **GROUP BY** agrupa por usuario. Toda la estadística la calcula la base, no el código de Node.

VISTA 4 · VW_USUARIOS_RESUMEN

CÓDIGO

```sql
CREATE OR REPLACE VIEW vw_usuarios_resumen AS
SELECT u.id, u.usuario, u.correo, u.nacimiento, u.creado_en,
       COUNT(h.id) AS total_partidas,
       MAX(h.puntaje) FILTER (WHERE h.puntaje IS NOT NULL) AS max_puntaje,
       MAX(h.jugada_en) AS ultima_partida
FROM usuarios u
LEFT JOIN historial_partidas h ON h.id_usuario = u.id
GROUP BY u.id, u.usuario, u.correo, u.nacimiento, u.creado_en;
```

LO QUE DIGO

Resume cada cuenta con sus estadísticas en **una sola consulta**. El `FILTER (WHERE ...)` calcula el máximo solo sobre puntajes no nulos, y el **LEFT JOIN** hace que también aparezcan los usuarios que todavía no jugaron.

04 · ESCRITURA ENCAPSULADA

Los 2 procedimientos almacenados

El backend los dispara con CALL; la lógica vive en el motor.

PROCEDIMIENTO 1 · SP_REGISTRAR_PARTIDA

CÓDIGO

```
CREATE OR REPLACE PROCEDURE sp_registrar_partida(
    p_id_usuario TEXT,
    p_zona       TEXT,
    p_puntaje    INTEGER,
    p_modo       TEXT,
    p_ganador    TEXT
)
LANGUAGE plpgsql AS $BODY$
BEGIN
    INSERT INTO historial_partidas
        (id, id_usuario, zona, puntaje, modo, ganador)
    VALUES (
        gen_random_uuid()::TEXT,
        p_id_usuario, p_zona, p_puntaje, p_modo, p_ganador
    );
END;
$BODY$;
```

LO QUE DIGO

Recibe los 5 datos del resultado y guarda **una fila en** `historial_partidas`. `gen_random_uuid()` genera el id único del registro. En mi juego lo invoca el endpoint `POST /api/procedimientos/registrar-partida` con un **CALL**.

PROCEDIMIENTO 2 · SP_CREAR_PARTIDA_COMPLETA

CÓDIGO

```
CREATE OR REPLACE PROCEDURE sp_crear_partida_completa(
    p_id_partida  TEXT,
    p_estado_json TEXT,
    p_id_usuario  TEXT,
    p_zona        TEXT,
    p_puntaje     INTEGER,
    p_modo        TEXT,
    p_ganador     TEXT
)
LANGUAGE plpgsql AS $BODY$
BEGIN
    INSERT INTO historial_partidas
        (id, id_usuario, id_partida, zona, puntaje, modo, ganador)
    VALUES (
        gen_random_uuid()::TEXT,
        p_id_usuario, p_id_partida, p_zona, p_puntaje, p_modo, p_ganador
    );

    INSERT INTO partidas (id, estado_json)
    VALUES (p_id_partida, p_estado_json);
END;
$BODY$;
```

LO QUE DIGO

> Este hace **dos inserciones en una sola operación**: primero el resultado en `historial_partidas` (relacionado por `id_partida`) y después la partida con su `estado_json` (el tablero, cartas encontradas, etc.). Como es una sola operación, es **atómico**: o se guardan las dos, o ninguna.

Cómo se llaman: PostgreSQL prohíbe subconsultas como argumento de `CALL`, por eso en las demos van dentro de un bloque `DO $$ ... $$` que primero resuelve el id del usuario y luego llama al procedimiento.

05 · AUTOMATIZACIÓN

El trigger de la tabla partidas

Recalcula la fecha de actualización solo, dentro de la base.

✅ **Sí se aplica.** El trigger se dispara en cada UPDATE real sobre `partidas`, sin que el backend lo tenga que hacer.

PARTE 1 · LA FUNCIÓN

CÓDIGO

```
CREATE OR REPLACE FUNCTION fn_partidas_actualizar_fecha()
RETURNS TRIGGER AS $$
BEGIN
    NEW.actualizada_en = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

LO QUE DIGO

Esta función es la que hace el trabajo. `NEW` es la fila que se está modificando y `now()` es la hora actual del servidor: le asignamos la fecha al campo `actualizada_en` y devolvemos la fila con `RETURN NEW` para que se guarde.

PARTE 2 · EL TRIGGER QUE LA ENGANCHA

CÓDIGO

```
CREATE TRIGGER trg_partidas_actualizar_fecha
    BEFORE UPDATE ON partidas
    FOR EACH ROW EXECUTE FUNCTION fn_partidas_actualizar_fecha();
```

LO QUE DIGO

BEFORE UPDATE significa que se ejecuta antes de cada modificación, y **FOR EACH ROW** que corre por cada fila afectada. Así, cada vez que se actualiza una partida, la fecha se refresca automáticamente.

PARTE 3 · CÓMO SE DEMUESTRA

CÓDIGO

```
UPDATE partidas SET estado_json = estado_json
 WHERE id = 'partida-demo';

SELECT id, creada_en, actualizada_en FROM partidas;
```

LO QUE DIGO

> Hago un UPDATE cualquiera y el `SELECT` muestra cómo `actualizada_en` cambió sola. El backend nunca escribió esa fecha: la puso el trigger.

06 · GUION DE DEMO

Los 5 pasos de la sustentación

Todo desde pgAdmin con datos reales.

1. **Cargar datos:** `npm run seed:demo` (llena tablas y vistas).
2. **ACID (1):** mostrar el `CALL` de `sp_crear_partida_completa` y mencionar el `BEGIN/COMMIT`.
3. **CAP (2):** explicar que es un monolito de un solo nodo → sistema CA.
4. **Vistas (3):** ejecutar los `SELECT * FROM vw_...`.
5. **Procedimientos (4):** ejecutar los `CALL sp_...`.
6. **Trigger (5):** hacer un `UPDATE` en `partidas` y ver cambiar `actualizada_en`.

Esquema: `sql/esquema.sql` · Consultas listas: `sql/consultas-demo.sql` · Guía: `sql/COMO-VER-EN-PGADMIN.md`.